
Chap 9, 10: Modern Scheduling

Proportional Share Scheduling

Multiprocessor Scheduling

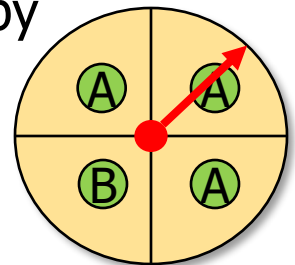
Proportional-Share (Fair-Share) Scheduler

- Instead of turnaround or response time, a scheduler tries to guarantee that each job obtain a certain percentage of CPU time. *turnaround, response time of only fair*

- **Lottery scheduling**

- The percent of tickets that a process has represents its share of the system resource.
- Example
 - Two processes, A (75 tickets) and B (25 tickets)
 - A to receive 75% of the CPU and B the remaining 25%.
- achieves this probabilistically (but not deterministically) by holding a lottery every time slice.
- Pick a winning ticket from 0 to 99
- Randomized approach *probability*
→ no guarantee
- How To Assign Tickets? Ticket-assignment problem

A=0~74
B=75~99



Open problem, 해결하기 쉽지 않음

Fairness

- **Fairness metric, F**

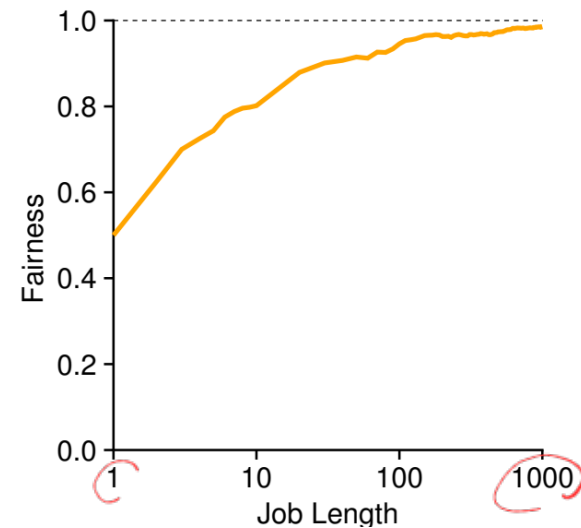
- The time the first job completes divided by the time that the second job completes.
- Same tickets (100) and same run time (R)
 - If the first job finishes at time 10 and the second job at 20,
 $F = 10/20 = 0.5$.
 - If $F=1$, perfect fairness

- **When the job length is not very long,**

- average fairness can be quite low.

- **For a good fairness,**

- Jobs must run for a significant number of time slices



Stride Scheduling

- Randomness is simple and approximately correct, but will not deliver the exact right proportions, especially over short time scales.
- **Stride scheduling** (Deterministic fair-share scheduler)
 - Each job in the system has a stride, which is inverse in proportion to the number of tickets it has.
 - Example
 - Jobs A, B, and C, with 100, 50, and 250 tickets, respectively
 - Compute the stride of each by dividing some large number by the number of tickets each process has been assigned.
 - A ($10,000/100 = 100$), B ($10,000/50 = 200$), C ($10,000/250 = 40$)
 - Pick the process to run that has the lowest pass value so far.
 - Every time a process runs, increment a counter for it (called its pass value) by its stride to track its global progress.

```
curr = remove_min(queue);    // pick client with min pass
schedule(curr);              // run for quantum
curr->pass += curr->stride;   // update pass using stride
insert(queue, curr);         // return curr to queue
```

Stride Scheduling

C ran five times, A twice, and B just once,
exactly in proportion to their ticket values of 250, 100, and 50.

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A <i>A 먼저 한라오 가라 (A, B, C)</i>
100	0	0	B <i>B 먼저 한라오 가라 (B, C)</i>
100	200	0	C
100	200	40	C <i>pass value가 가장 낮은 걸 실행</i>
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	... 다음으로 돌아가 반복
<i>2 100</i>	<i>1 200</i>	<i>5 40</i>	

Comparison between lottery and stride schedulers

Lottery scheduling has no global state per process.

easy to incorporate new processes in a sensible manner.

*선배 따라 걸라
proportion을 맞출*

정확하게 비율을 맞출

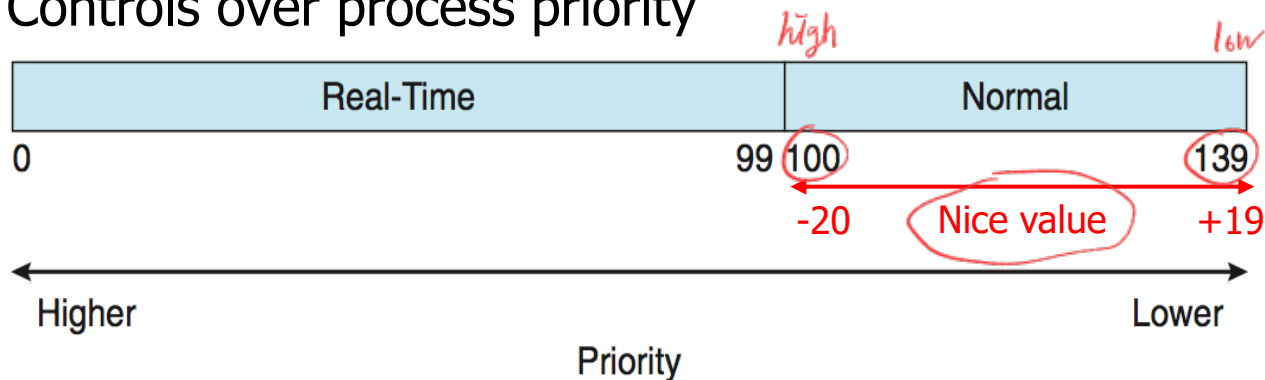
*새 process가 들어오면
pass값을 설정해줘야함*

Scheduling in Linux

- **Version 2.6.23+**

- CFS (Completely Fair Scheduler)

- Fairly divide a CPU evenly among all competing processes.
 - Highly efficient and scalable spending very little time making scheduling decisions
 - Uses red-black trees, Not based on run queues
 - Improvement in interactive performance of the scheduler
 - $O(\log N)$ scheduling
 - N: number of tasks in the runqueue (red-black tree)
 - Controls over process priority



CFS (completely fair scheduler)

Virtual runtime

- **vruntime** ($\approx$ pass value in stride scheduling)
 - As each process runs, it accumulates vruntime.
 - When a scheduling decision occurs, pick the process with the lowest vruntime to run next.
- **sched_latency**
 - determine how long one process should run before considering a switch (time slice).
 - if $\text{sched_latency} = 48\text{ms}$ and 4 processes running, the per-process time slice is 12ms.
- **min_granularity**
 - never set the time slice of a process to less than this value, ensuring that not too much time is spent in scheduling overhead.
 - if 10 processes running, the time slice of each process is 6ms/ instead of 4.8ms.

$$\text{time slice} \times \frac{\text{sched_latency}}{\text{\#process}}$$

이 시간 동안 모든 process가 3회 1번 schedule 시작한다

→ 같은 priority

이 밑으로는 time slice 줄이지 않음

따라서 scheduling으로 인한 overhead

CFS (completely fair scheduler)

- Weighting (Niceness)**

- Time slice depends on weight, not priority

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency}$$

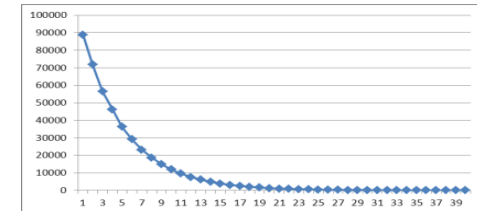
total weight

```
static const int prio_to_weight[40] = {
    /* -20 */ 88761, 71755, 56483, 46273, 36291,
    /* -15 */ 29154, 23254, 18705, 14949, 11916,
    /* -10 */ 9548, 7620, 6100, 4904, 3906,
    /* -5 */ 3121, 2501, 1991, 1586, 1277,
    /* 0 */ 1024, 820, 655, 526, 423,
    /* 5 */ 335, 272, 215, 172, 137,
    /* 10 */ 110, 87, 70, 56, 45,
    /* 15 */ 36, 29, 23, 18, 15,
};
```

- A (nice = -5), B (nice = 0), sched_latency = 48ms**

- time_slice_A = 48 * 3121 / (3121 + 1024) = 48 * 3/4 = 36ms
- time_slice_B = 48 * 1024 / (3121 + 1024) = 48 * 1/4 = 12ms
- What if A (nice = 5) and B (nice = 10)?
 - 110 / (110 + 335) = 25%
 - CPU proportionality ratios are preserved when the difference in nice values is constant

kernel/sched/sched.h



- Uses **virtual runtime** to track the progress of each process *i*

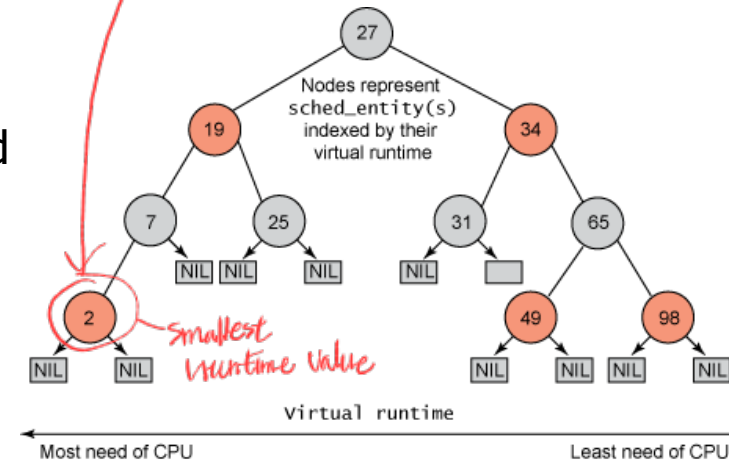
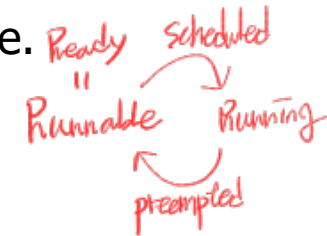
$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i$$

1024

- runtime_i: the amount of execution time of process i (equals to slice if there is no preemption from higher priority task).
- weight₀/weight_i (≈ stride value in stride scheduling)
 - weight₀: the default weight of 1024
 - high as a task consumes time-slice; low as a task has a higher priority

CFS (completely fair scheduler)

- Scheduler has to find the next job to run as quickly as possible.
 - Simple data structures like lists don't scale.
- Uses a vruntime-based **red-black tree** for each CPU
 - Red-black tree is a self-balancing binary tree
 - All runnable tasks are sorted in the RB tree by vruntime
 - If a process goes to sleep, it is removed from the tree.
 - The leaf at the leftmost inside RB tree has the smallest value and most entitled to run at any given time.
 - Operations (insertion and deletion) are logarithmic in time, i.e., $O(\log n)$
- Dealing with I/O and sleeping processes
 - Problem
 - Two processes, A (runs continuously) and B (has gone to sleep for 10 seconds).
 - When B wakes up, its vruntime will be 10 seconds behind A's.
 - B will now monopolize the CPU for the next 10 seconds while it catches up.
 - CFS sets the vruntime of B to the minimum value found in the tree.



Multiple Processor Scheduling

프로세서가 여러 개일 때

- **Asymmetric multiprocessing**

- A single special processor, called master server, handles all scheduling decisions, I/O processing, and other system activities
- The other processors execute only user codes
- Simple

- **Symmetric multiprocessing (SMP)**

- Each processor: self-scheduling
 - Two types of ready queue
 - Single-Queue Scheduling: Global (common) ready queue
 - Multi-Queue Scheduling: Private queue for each processor
- 

Single-Queue Scheduling (SQMS)

- Putting all jobs that need to be scheduled into a single queue.

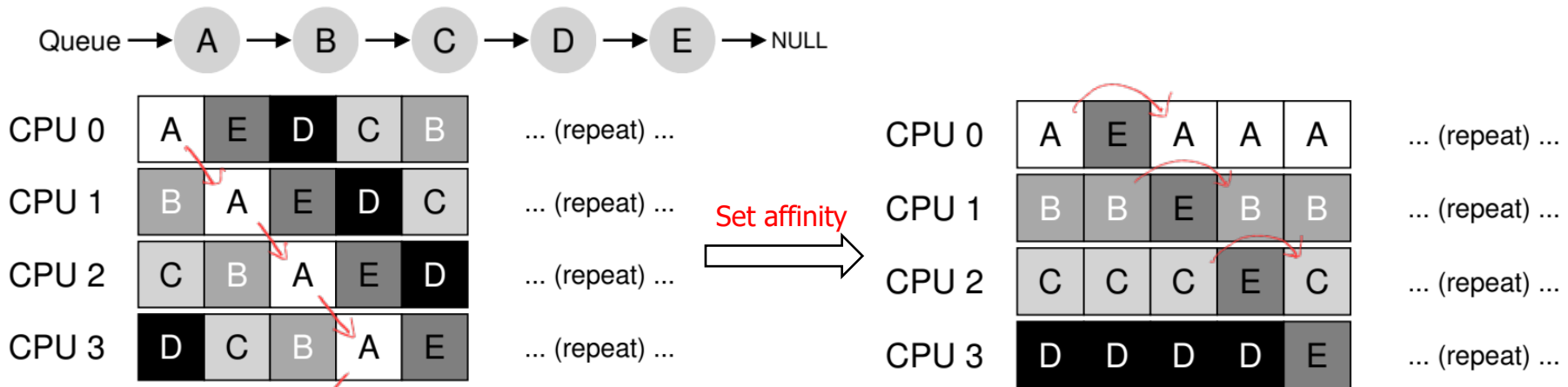
+ simplicity

- lack of scalability

- Each CPU must acquire the lock to access the single queue.

- cache affinity

- Need to use an affinity mechanism to make it more likely that process will continue to run on the same CPU if possible.



Each task is bounced from CPU to CPU.

각각 다른 CPU에서 실행 (cache affinity ↓)

Only E migrates from CPU to CPU.

같은 CPU에서 계속 실행

Single-Queue Scheduling (SQMS)

- Cache affinity (Processor affinity)
 - Avoid migration of processes from one processor to another and attempt to keep a process on the same processor
 - To reduce the cost of invalidating and repopulating caches and TLBs
 - Soft affinity
 - Attempts to keep a process on the same processor, but does not guarantee it
 - Hard affinity
 - No migration of processes
 - Linux has system calls and commands that support hard affinity
 - # taskset -pc 0-2,4-7 35324
 - # taskset -p 0x000000F7 35324

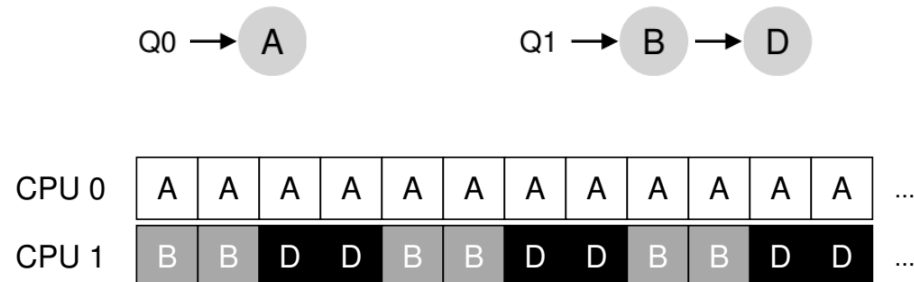
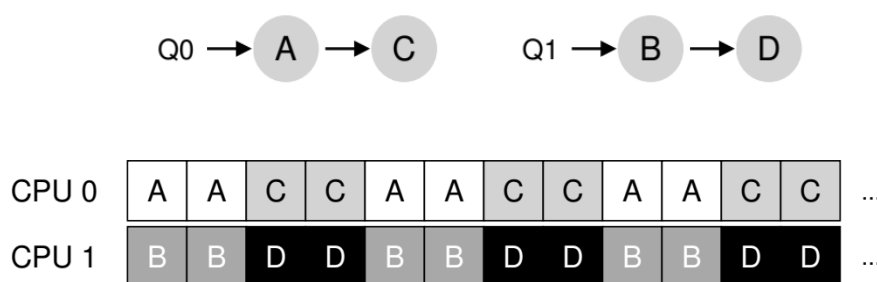
```
#include <stdio.h>
#include <stdlib.h>
#define __USE_GNU
#include <sched.h>
#define MAX_CORES 8
int main()
{
    cpu_set_t mask;
    int core;

    /** Write Affinity */
    CPU_ZERO(&mask);
    for(core=0; core < MAX_CORES; core++)
        CPU_SET(core, &mask);
    CPU_CLR(3, &mask);
    sched_setaffinity(0, sizeof(mask), &mask);

    /** Read Affinity */
    sched_getaffinity(0, sizeof(mask), &mask);
    printf("Process %d is allowed to run on core: ", getpid());
    for(core=0; core < MAX_CORES; core++){
        if(CPU_ISSET(core, &mask) == 1)
            printf("%d ", core);
    }
    return 0;
}
```

Multi-Queue Scheduling (MQMS)

- When a job enters the system, it is placed on exactly one scheduling queue, according to some heuristic (e.g., random, or picking one with fewer jobs than others).
- Scheduled essentially independently
- **More scalable**
 - avoid the problems of information sharing and synchronization found in the single-queue approach.
- Intrinsically provides cache affinity
 - jobs stay on the same CPU
- Load imbalance problem *특정 CPU에 과중이 발생*



Imbalance

Multi-Queue Scheduling (MQMS)

- **Load balancing**

- Attempts to keep the workload evenly distributed across all processors in the system
- Push migration
 - A specific task that monitors the load on each processor
 - When it finds imbalance
 - Evenly distributes the load by moving (pushing) *작은 쪽으로 보낼*
processes from overloaded to underloaded processors
- Pull migration (work stealing)
 - Idle processor pulls a waiting processes from a busy *많은 쪽에서 가져옴*
processors

Linux Multiprocessor Schedulers



- **O(1) scheduler**

- Multiple queues *MQMS, CPU per queue*
- Priority-based scheduler (similar to the MLFQ)

- **CFS** *priority → weight → time slice
→ runtime* *2차원 2차원 priority scheduler*

- Multiple queues *CPU per queue*
- Deterministic proportional-share approach (more like Stride scheduling)

- **BFS**

- Single queues *SQMS* *하나의 queue*
- Proportional-share, but based on a more complicated scheme known as Earliest Eligible Virtual Deadline First (EEVDF) *Linux*

*EEVDF의 개념,
가능과 안됨*

Quiz

1. Stride scheduling is a proportional-share scheduling algorithm. (T)
2. In stride scheduling, the task with the ~~highest~~ pass value is chosen next. (F)
3. In CFS, a task with a lower weight (higher nice value) receives a smaller share of CPU time in CFS. (T) *= priority ↓*
4. Unlike O(1) schedulers, CFS does not assign fixed time slices to tasks. (T)
5. In CFS, a task with lower virtual runtime gets scheduled before one with a higher virtual runtime. (T)
6. The ~~sched~~_latency controls the maximum amount of time a task can run before being preempted. (F) *weight*
7. The minimum time slice a task can receive is determined by min_granularity. (T)
8. While a task is waiting for I/O completion, its vruntime continues to increase. (F)
9. Load balancing only occurs during process creation or termination. (F)
10. The command "taskset -p 0x000000A0 1234" assigns CPU 0 and 2 to PID 1234. (F)
*1010 0100
⇒ 7,5*

Scheduling Example in CFS

- **The system has one CPU.**
- **sched_latency: 30ms, min_granularity : 3ms**
- **There are three processes: P1, P2, and P3.**
 - P1, P2, and P3 require 15ms, 10ms, and 10ms execution time, respectively.
- **All processes are runnable and have different nice values:**
 - P1: nice = 0 $\rightarrow$ weight = 1024
 - P2: nice = 5 $\rightarrow$ weight = 335
 - P3: nice = 10 $\rightarrow$ weight = 110
- **Draw the Gantt charts for process scheduling. (Ignore the context switch overhead.)**

Scheduling Example in CFS

- **Step 1: Compute Total Weight and Time Slices**

- Total weight = $1024 + 335 + 110 = 1469$
- We compute each task's time slice as:
 - P1: $(1024 / 1469) \times 30\text{ms} \approx 20.9\text{ms}$
 - P2: $(335 / 1469) \times 30\text{ms} \approx 6.8\text{ms}$
 - P3: $(110 / 1469) \times 30\text{ms} \approx 2.3\text{ms}$

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency}$$

Scheduling Example in CFS

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

• Step 2: Schedule Processes Based on vruntime

- Each process starts with an initial vruntime = 0.
- CFS selects the task with the smallest vruntime to run first.

Time	Process	vruntime
0	P1 (15) -> finish	$0 + (15) \times 1024/1024 = 15$
15	P2 (6.8)	$0 + (6.8 \times 1024/335) \approx 20.8$
21.8	P3 (3)	$0 + (3 \times 1024/110) \approx 27.9$
24.8	P2 (3.2) -> finish	$20.8 + (3.2 \times 1024/335)$
28	P3 (3)	$27.9 * 2$
31	P3 (3)	$27.9 * 3$
34	P3 (1) -> finish	$27.9 * 3 + (1 \times 1024/110)$
35		

P1: 20.9ms
P2: 6.8ms
P3: ~~2.3ms~~ 3ms

Gantt Chart of Process Scheduling under CFS with min_granularity

